

Easy Learned Caching: An Honest Look at Augmenting S3-FIFO with a Learned Promotion Gate

Minkai Li
Harvard University

Abstract

S3-FIFO and SIEVE have shown that simple FIFO-based caches with quick demotion and lazy promotion can match or beat much more sophisticated LRU-based heuristics. In parallel, deployed learned policies such as HALP demonstrate meaningful gains from augmenting heuristic structures with small online models. We’re interested in exploring learning in caching, and use a case study of a very simple augmentation—replacing S3-FIFO’s hardcoded fixed-threshold promotion rule with a tiny online linear classifier—to ask whether such an augmentation is worth its cost. We ignore latency and other production concerns, but we deliberately evaluate a simple, interpretable model that could plausibly be deployed, rather than higher-scoring but less interpretable architectures that production systems will not adopt.

We implement S3-FIFO-L, an online logistic-regression promotion gate over three cache-internal features (log-hits, age in S , recency-since-last-hit) trained against a Belady-derived binary label. Against vanilla S3-FIFO at the published default, the gate wins across our initial test set. Against a fully-tuned S3-FIFO baseline (the post-hoc-best $(|S|, T)$ per workload), the gate ties or loses on 23 of 28 cells with mean net effect $+0.39$ pp request-miss-ratio. However, despite this net loss, on three cells where the gate beats tuned S3-FIFO by ≥ 0.5 pp, it also beats every other classical and learned baseline available—including on workloads where tuned S3-FIFO is already the best non-oracle policy, demonstrating the real potential of learned caching. We pair this with a diagnosis of two failure modes—label-horizon misalignment and S3FIFO’s initial struggles—that explain where simple learned promotion gates can and cannot help. The contribution is the characterization that a learned promotion gate is a narrow but real augmentation lever for improving caching.

1 Introduction

Production caches rely on heuristics. LRU, FIFO, S3-FIFO, SIEVE, and LFU are each, in effect, a small hand-coded classifier: “promote on hit,” “evict the tail,” “admit on miss.” In

parallel, deployed learned policies in industry have shown that augmenting these heuristic structures with small models can deliver meaningful gains. HALP, deployed on YouTube’s CDN, achieves a 9.1% byte-miss-ratio reduction at 1.8% CPU overhead by adding a small continuously-trained MLP that re-ranks the last $k = 4$ eviction candidates selected by an LRU-based heuristic [3]. The model is small, the features are few, and the heuristic stays in charge of the search space. The learner only re-orders. In parallel, recent work in FIFO-based caches has shown that very simple structural changes, notably *quick demotion* (the observation that most objects are accessed only once, so a probationary queue should evict them quickly) and *lazy promotion* (the observation that promotion work can be batched at eviction time rather than performed at every access), suffice to match or beat much more sophisticated LRU-based designs [1, 2].

In studying the potential of using learning in caching, this project uses the case study of S3-FIFO to analyze the potential of learning in new FIFO-based caches, namely on the binary classifier that decides whether a hit-on- S (the probationary queue) object gets promoted to M (the main queue). S3-FIFO’s existing rule is a fixed-threshold rule: hit-once/twice $\Rightarrow$ promote, otherwise demote. This is coarse. Replacing it with an online linear model adds at most a few multiply-adds per insertion and a handful of weights, while adding significant flexibility depending on the workload.

We follow the literature-augmented-heuristic philosophy of HALP and build what we call **S3-FIFO-L**: a learned promotion gate running entirely on cache-internal state and trained online against a Belady-derived binary label. We evaluate it across 14 production traces from Twitter, MSR, Alibaba, CloudPhysics, Wikipedia, and Meta CDN/storage, against canonical caching baseline and a static-tuning baseline that we argue is the *correct* comparison.

2 Background and Related Work

2.1 S3-FIFO and lazy promotion

S3-FIFO [1] maintains three queues: a small probationary FIFO S holding $\sim 10\%$ of cache capacity, a main FIFO M holding the remaining $\sim 90\%$, and a ghost queue G tracking recently evicted object IDs. New objects are admitted to S . On a hit in S , the accessed bit(s) are incremented; at the time the object reaches the tail of S , if the bit(s) are above some threshold T (generally 1 or 2), the object is promoted to M or demoted to G (if not). M uses a similar visited-bit rule for its own re-insertion decisions. Ghost-hits in G admit the returning object directly to M .

The design is built around two principles: *quick demotion*, which gives one-hit wonders a single short residency window before they are removed, and *lazy promotion*, which batches all promotion work to eviction time, removing the per-access, locking updates that LRU-based caches require.

2.2 Three paradigms of learned caching

Our literature review surfaced three structural patterns in learned cache replacement, broadly tracking the trade-off between expressive power and per-decision overhead.

Expert weighting. The earliest and lightest paradigm. LeCaR [6] runs shadow LRU and LFU policies in parallel and uses regret minimization to combine their decisions, with ghost-cache hits providing the loss signal. CACHEUS [7] extends this with two more principled experts (scan-resistant LRU and churn-resistant LFU) and learns the learning rate itself via stochastic hill-climbing. Across 17,766 experiments on 329 workloads, CACHEUS is the most consistently competitive policy in its benchmark. The cost is two parallel shadow policies; there is no per-object feature engineering.

Object-level Belady approximation. The most expressive paradigm. LRB [4] trains a LightGBM model on ~ 44 per-object features—inter-request time deltas, exponentially decayed frequency counters at multiple timescales, object size—to predict next-access time. The object predicted farthest in the future is evicted. On six production CDN traces, LRB reduces WAN traffic by 5–24%.

Group-level learning. The middle path. GL-Cache [5] clusters objects by insertion-time proximity and learns a utility function per group via online XGBoost. Eviction happens at the group level. Per-object metadata drops below 1 byte, throughput is $228\times$ that of LRB, and hit-ratio improves by 7% on average across 118 traces.

2.3 HALP: the deployed template

HALP [3] is the closest deployed analog of what we build. At YouTube’s CDN, an LRU-class heuristic pre-selects $k = 4$ eviction candidates from the tail, and a small continuously-trained MLP scores each candidate. Pairwise preference loss

Feature	Definition
$\log(1 + \text{hits})$	uncapped log-count of hits in S
age_S	$(\text{now} - t_{\text{insert}})/ S $
rec	$(\text{now} - t_{\text{lasthit}})/ S $

Table 1: The three V4 features. Times are in virtual units (request counts). Normalization by $|S|$ keeps the features scale-invariant under cache resizing.

against Belady-derived “correct” pairs is the training signal. The heuristic stays in charge of the candidate set; the model only re-ranks. The reported gain is 9.1% byte-miss-ratio reduction at 1.8% CPU overhead.

We borrow three things from HALP: (i) the philosophy of leaving the heuristic in charge of the search space, (ii) the use of Belady-derived labels rather than reinforcement-style rewards, and (iii) the continuous-online-update training pattern. We diverge in three ways: (i) we apply the model at the *promotion* decision rather than the eviction decision, (ii) we use an online linear model rather than an MLP, and (iii) we use cache-internal features only, with no per-object metadata beyond a few floats.

3 Design

3.1 Decision rule

S3-FIFO-L modifies exactly one decision in S3-FIFO: the $S \rightarrow M$ promotion classifier. At the time an object reaches the tail of S , instead of consulting the accessed bit alone, we score

$$z = b + w_1 \cdot \log(1 + \text{hits}) + w_2 \cdot \text{age}_S + w_3 \cdot \text{rec},$$

and promote iff $\sigma(z) > 0.5$ ($z > 0$). The features are defined in Table 1; all are computable from state already maintained by S3-FIFO plus three additional floats per S -resident.

3.2 Online training

The training label is a Belady-binary indicator:

$$y = \mathbf{1}[\text{nxt_acc_vtime}(x) - \text{now} < H],$$

with horizon $H = \text{cache size}$. We use the `next_access_vtime` field already present in libCacheSim’s `oracleGeneral` trace format, so no extra trace processing is required. Updates use online stochastic gradient descent on the logistic loss with learning rate $\eta = 0.05$ and L2 regularization $\lambda = 10^{-4}$. In a production deployment, the Belady label would not be available at decision time. Following HALP’s delayed-online-update pattern, the production version of the gate would: (i) snapshot $(\text{obj_id}, \phi(x), \text{gate output})$ into a pending buffer at each promotion decision; (ii) wait for the label to resolve via either a hit on x within the horizon ($y = 1$)

or eviction of x from M or G without a re-access ($y = 0$); (iii) apply one SGD step on the resolved tuple and drop it. The pending buffer size is bounded by the cache’s residency window. For this paper, we use offline Belady labels directly as an upper bound on what the delayed-online scheme would learn.

3.3 Feature ablation: from V3 to V4

The headline feature set above (V4) is the result of a systematic ablation. An earlier version (V3) used four features: $\log(1 + \text{hits})$, the 2-bit frequency counter S3-FIFO already maintains, age_S , and recency since last hit. The 2-bit frequency counter is redundant with $\log(1 + \text{hits})$, which is the same quantity uncapped, and I saw no loss in accuracy from dropping these feature. Removing this feature produces V4: 3 features, 4 parameters. V4 ties or beats V3 on every cell in our evaluation. We also tried a piecewise-linear GAM extension (V9) with 16 parameters; it ties or loses to V4 and overall seemed to be much more finicky in training. Overall, early experiments suggested that the Belady boundary is well-approximated by a linear function of the V4 features at the trace lengths we run (500K requests).

4 Implementation

4.1 Harness architecture

We build on libCacheSim [11], a high-performance C++ cache simulator that ships canonical implementations of 18 baselines: classical replacement policies (FIFO, LRU, LFU, ARC, TwoQ, SLRU, SIEVE, ClockPro, S3-FIFO), recency-based non-LRU policies (LIRS, Hyperbolic), frequency-aware policies (WTinyLFU, GDSF, LHD, QDLP), the Belady-optimal oracle, and the learning-augmented policies LeCaR and CACHEUS. Our harness has two components.

Native baseline path. The standalone `cachesim` binary is invoked via a Python subprocess wrapper. Per-algorithm wall-clock throughput at cache size 10,000 is roughly 7–10M requests/second on a 500K-request trace; at cache size 1,000 the same algorithms run at ~ 20 M requests/second. **Python plugin path.** V4 and its ablations run in pure Python via libCacheSim’s `PluginCache` interface, with `init/hit/miss/eviction/remove/free` hooks. This is slower but extensible: each ablation is a ~ 100 -line subclass.

4.2 Trace selection

We evaluate on 14 traces from the libCacheSim public dataset collection, spanning four workload families:

- **Twitter KV** (5 traces): cluster10, 26, 45, 50, 52. Key-value memcached traces released alongside the OSDI

Trace	Cache	S3-FIFO	V4	Δ
twitter c52	1,000	0.3355	0.3184	−1.71
twitter c52	10,000	0.2110	0.2078	−0.32
twitter c45	1,000	0.5630	0.5459	−1.71
twitter c45	10,000	0.4982	0.4940	−0.42
msr hm_0	1,000	0.3715	0.3704	−0.11
cloudphys.	5,000	0.7482	0.7365	−1.17

Table 2: V4 vs. vanilla S3-FIFO at $|S| = 10\%$, $T = 1$. All deltas are in percentage points of request-miss-ratio. Negative is better.

’20 Twitter workload analysis [12]. Cluster52 is the S3-FIFO paper’s canonical hardest trace; clusters 45 and 50 are write-heavy; cluster26 is locality-rich; cluster10 is near-uniform.

- **Block I/O** (6 traces): MSR Cambridge `hm_0`, `proj_0`, `prxy_0`; CloudPhysics; Alibaba 110, 185. `prxy_0` has very high locality (Belady ≈ 0.04 at cache size 10K) and acts as a stress test where the right policy is closer to LFU.
- **CDN** (2 traces): Wikipedia CDN 2019 and Meta CDN `reag`.
- **Storage** (1 trace): Meta Tectonic `block1`.

All traces are in libCacheSim’s `oracleGeneral` format, with the `next_access_vtime` field used for both Belady oracle computation and our V4 training labels. Two of the local CSV traces (Twitter cluster52 and CloudPhysics) required a one-time conversion script to add `next_access_vtime` via a backward pass over the trace.

We evaluate two cache sizes per trace, chosen to roughly bracket the working-set knee (typical cache-fraction bands are 0.1–1% and 1–10% of the unique-object footprint). Each cell uses 500K requests; smaller traces use the full file.

5 First Results: V4 vs. Vanilla S3-FIFO

We first evaluate V4 against vanilla S3-FIFO at $|S| = 10\%$, $T = 1$ split. Table 2 reports the six initial test cells. The pattern is encouraging: V4 wins on every cell, by 0.1–1.7 pp request-miss-ratio.

The wins span workload families: Twitter KV (clusters 52 and 45), MSR block I/O, and CloudPhysics.

The trained weights are diagnostically distinct per cell, evidence the model is learning real workload structure rather than memorizing one. On Twitter cluster52 at cache size 10,000, the converged weights have $w_{\log \text{hits}} = +4.61$ and the gate effectively uses uncapped hit count to decide. On CloudPhysics 5,000, $w_{\text{rec}} = -1.08$ dominates, reflecting that recency-since-last-hit is the load-bearing feature on this temporally-clustered

block-I/O trace. On Twitter cluster45 at cache size 1,000—a write-heavy trace—the model converges to an effectively No-Promote policy ($z_{\max} \approx -0.68$ across all decisions); the right policy is to never promote, and the gate auto-discovers this.

6 The Full Result: Is Learning Usable?

This initial result materialized somewhat early in the project’s lifetime and showed the promise of applying learning to caching. However, as I expanded the project to involve testing over a greater number of traces, it also made sense to think more critically about the baselines I was comparing against. $|S| = 10\%$ is the S3FIFO’s default, but the paper itself acknowledges that this split is suboptimal on some workloads. I would assume that in cases where a production team is looking to apply some learning to their caching system, they’re also willing to tune the basic hyperparameters of S3FIFO first. A proper comparison should let S3-FIFO use the ideal hyperparameters for the cache. This seems like a more reasonable and more effective baseline to compare S3FIFO-L on.

We therefore introduce two additional baselines:

- **OptS_T2**: the post-hoc-best of S3-FIFO at $T = 2$ with $|S| \in \{0.01, 0.05, 0.10, 0.25\}$. One parameter and four configurations.
- **OptST**: the post-hoc-best of S3-FIFO over $|S| \in \{0.01, 0.05, 0.10, 0.25\}$ and $T \in \{1, 2, 3\}$. Two parameters and 12 configurations.

Both OptS_T2 and OptST are post-hoc oracles: they pick the right parameters with full knowledge of the trace, which is unfair to the gate (a true online learner). However, in production settings, I’d imagine that deriving the optimal parameters is something that can be quickly back-tested and should remain fairly static over time, except in cases of major distributional shift in workload (at which point, new values can be chosen).

For the sake of balance, we also add in a new learned model, V4+OptS. This model uses the V4 learning configuration but will run the configuration over $|S| \in \{0.01, 0.05, 0.10, 0.25\}$, selecting the best post-hoc model for its performance. This emulates the fact that if a production team were to use learning in their cache system, they would also want to quickly tune the last remaining cache hyperparameter ($|S|$ size).

6.1 Results across 14 traces

Table 3 reports the cross-trace win/loss tally. The wins-threshold is 0.1 pp.

The decisive comparison is V4+OptS versus OptST: 5 wins, 10 losses, 13 ties, with a mean Δ of +0.39 pp. *The 4 trainable parameters of V4 contribute do not improve on average over a static $|S|$ and T grid search.*

Comparison	W	L	T	mean Δ
V4 vs. OptST	3	16	9	0.98
V4+OptS vs. OptS_T2	8	7	13	0.15
V4+OptS vs. OptST	5	10	13	0.39

Table 3: Win/loss tally on 28 cells. Mean Δ in pp; positive favors the second name. The decisive row is the bolded one: V4+OptS vs. OptST.

6.2 Three cells where learning genuinely helps

V4+OptS still beats OptST by at least 0.5 pp on three cells:

- **cloudphysics 5,000**: V4+OptS 0.7333 vs. OptST 0.7441 (−1.08 pp). Recency-since-last-hit is the load-bearing feature ($w_{\text{rec}} = -1.08$); no static ($T, |S|$) combination can express “promote when recency is short.”
- **msr_proj_0, cache 10,000**: V4+OptS 0.3673 vs. OptST 0.3771 (−0.98 pp). On this block-I/O workload the gate exploits a frequency-by-recency interaction that no static knob combination captures.
- **twitter cluster52, cache 1,000**: V4+OptS 0.3133 vs. OptST 0.3223 (−0.90 pp). The gate converges to an No-Promote policy ($z_{\max} < 0$), identifying that the S3FIFO policy over-promotes items that can appear in bursts. No static ($T, |S|$) can reproduce this without being hand-set to skip promotion entirely.

One of the most exciting results is that these traces are not ones in which OptST does poorly relative to other baselines – in fact, it is the best on cloudphys and msr and 2nd to TwoQ on twitter_52). This means that V4+OptS beats all baselines on these traces. Unfortunately, I only discovered this property on the last day of the project, as I finish this writeup. However, this suggests an exciting research direction of learning augmentations to S3FIFO. The learned gate is not just boosting S3FIFO on traces where it is poor, allowing the learned cache to beform better (relative to S3FIFO) but to obtain middling results relative to other baselines. Instead, it suggests that there are cases where S3FIFO is already the best policy (and would perhaps be the choice for a production system) and that augmenting with simple learning can improve this system even more.

6.3 One cell where learning fails catastrophically

msr_prxy_0 at cache size 1,000 is the failure case. V4+OptS has a miss ratio of 0.5295 whereas OptS_T2 has 0.4472; the best classical baseline (WTinyLFU) has 0.3733. V4+OptS is +8.23 pp worse than the static-tuning ablation and +15.62 pp worse than the best classical contender.

Reuse distance	Reuses	% of reuses	Cumulative %
< 64	748,131	6.1%	6.1%
[64, 128)	91,611	0.7%	6.8%
[128, 256)	143,558	1.2%	8.0%
[256, 512)	217,094	1.8%	9.8%
[512, 1024)	654,214	5.3%	15.1%
<i>cache boundary (15.0% of reuses hit; 85.0% miss)</i>			
[1024, 2048)	6,134,361	49.6%	64.7%
[2048, 4096)	1,824,103	14.8%	79.5%
[4096, 8192)	1,135,527	9.2%	88.7%
[8192, 16384)	522,164	4.2%	92.9%
[16384, 32768)	260,135	2.1%	95.0%
[32768, 65536)	146,965	1.2%	96.2%
[65536, 131072)	112,979	0.9%	97.1%
[131072, 262144)	53,725	0.4%	97.5%
$\geq 262,144$	318,720	2.6%	100.0%
Total reuses	12,363,287	100.0%	

Table 4: Reuse-distance distribution on `msr_prxy_0` (12.4M reuses with a future access; 1.2% of records have no future reuse and are excluded). Buckets are powers of two in number of intervening requests. The [1024, 2048) bucket alone holds 49.6% of reuses — objects re-arrive just past the cache boundary at size 1,000. The Belady label with $H = 1,000$ marks the entire [1024, ∞) tail as $y = 0$, training the gate to demote exactly the objects that will return at distance $\sim 1,000$.

The mechanism is visible in the reuse-distance distribution (Figure 4). 49.6% of all reuses fall in the [1024, 2048) bucket — objects re-arrive immediately outside the cache. A further 27.8% land in [2048, 16384). Only 15.0% of reuses are within the cache, so 85% of all reuses miss; more than half of those would hit if the cache were doubled. The Belady-binary label with horizon $H = 1,000$ classifies the entire [1024, ∞) tail as $y = 0$, training the gate to demote exactly the objects that will return at distance $\sim 1,000$.

This is a label-misalignment failure, not a feature failure or a capacity failure. In essence, the linear model and the labeling are not built to handle the issue of churning in caching, and the specific labeling decision exacerbates this issue. While this is a major issue, it is unclear if this is a problem that the learned gate should be particularly concerned with. OptST performs poorly on this trace relative to other classical baselines, and would not be selected for production in practice.

6.4 Where S3-FIFO itself is the wrong substrate

Beyond `prxy_0`, two cells reveal that the S3-FIFO substrate itself is mismatched to the workload regardless of the promotion gate.s

On `cluster26` at cache size 1,000, LRU achieves 0.1044 while V4+OptS achieves 0.1425. The trace is locality-rich

enough that pure recency dominates: LRU, ARC (0.1069), and LeCaR (0.1075) all beat every S3-FIFO variant. Even OptST only achieves 0.1272 — still +2.28 pp behind LRU. V4+OptS does beat the original S3-FIFO (−3.88 pp but cannot close the substrate gap; it ranks 11th of 17 non-Belady algorithms on this cell).

On `block1` (Meta storage) at cache size 1,000, ARC gets 0.5445 while V4+OptS gets 0.5526. OptST gets 0.5468, itself +0.23 pp behind ARC. V4+OptS shows improvement but still is 9th of 17 non-Belady algorithms.

These are not failures of the learned gate but failures of the substrate: no S3-FIFO configuration, learned or statically tuned, reaches the right answer.

7 Future Work

Adaptive label horizons. The single most concrete fix suggested by our failure analysis is to choose H from the trace’s reuse-distance distribution rather than from the cache size. A horizon at the 75th percentile of in-cache reuses, or a soft label would directly address the `msr_prxy_0` failure mode. **Apply the gate at the M -eviction decision.** S3-FIFO has a second hardcoded binary classifier at the M -eviction boundary (the $\text{freq} \geq 1$ rule, with reinsertion-with-decrement on the positive branch). A similar learned approach on this eviction boundary could perhaps yield complementary wins. **Production-realistic delayed online updates.** The current results use offline Belady labels as an upper bound on what the delayed-online HALP-style update scheme would learn. A next step is to implement the delayed-update pipeline—snapshot at decision time, resolve at hit-or-eviction, SGD on the resolved tuple—and measure the gap to the offline upper bound.

8 Conclusion

We tested whether replacing S3-FIFO’s hardcoded promotion rule with a tiny online linear classifier delivers reliable improvement across heterogeneous workloads. Against vanilla S3-FIFO the answer looked like yes; against fully-tuned S3-FIFO, the answer is mostly no, with a mean net effect that mildly favors static tuning.

What survived the ablation is what makes the result interesting. On three cells, the gate beats not just every static S3-FIFO configuration but every classical and learned baseline we evaluated—including on workloads where tuned S3-FIFO is already the best non-oracle policy. These are precisely the cells where the optimal policy is conditional on a cache-internal feature. This empirical result shows the struggles and potentials of finding a useful learning-augmented caching contribution: narrow and difficult to identify, but an opportunity for further exploration.

References

- [1] Juncheng Yang, Yazhuo Qiu, Yao Zhang, Yue Yue, and K. V. Rashmi. FIFO queues are all you need for cache eviction. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, 2023.
- [2] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. SIEVE is simpler than LRU: an efficient turn-key eviction algorithm for web caches. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*, 2024.
- [3] Zhenyu Song, Daniel S. Berger, Kai Li, Anees Shaikh, Wyatt Lloyd, Soudeh Ghorbani, Changhoon Kim, Aditya Akella, Arvind Krishnamurthy, Emmett Witchel, and Radhika Mittal. Cache eviction in practice: a heuristic-augmented learned policy. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*, 2023.
- [4] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed Belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*, 2020.
- [5] Juncheng Yang, Ziming Mao, Yao Yue, and K. V. Rashmi. GL-Cache: group-level learning for efficient and high-performance caching. In *21st USENIX Conference on File and Storage Technologies (FAST '23)*, 2023.
- [6] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. Driving cache replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage '18)*, 2018.
- [7] Liana V. Rodriguez, Farzana Yusuf, Steven Lyons, Eysler Paz, Raju Rangaswami, Jason Liu, Ming Zhao, and Giri Narasimhan. Learning cache replacement with CACHEUS. In *19th USENIX Conference on File and Storage Technologies (FAST '21)*, 2021.
- [8] Daniel S. Berger, Ramesh K. Sitaraman, and Mor Harchol-Balter. AdaptSize: orchestrating the hot object memory cache in a content delivery network. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI '17)*, 2017.
- [9] Nimrod Megiddo and Dharmendra S. Modha. ARC: a self-tuning, low overhead replacement cache. In *2nd USENIX Conference on File and Storage Technologies (FAST '03)*, 2003.
- [10] Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. In *35th International Conference on Machine Learning (ICML '18)*, 2018.
- [11] Juncheng Yang, Ziming Mao, Yao Yue, and K. V. Rashmi. libCacheSim: a high-performance cache simulator. <https://github.com/cacheMon/libCacheSim>.
- [12] Juncheng Yang, Yao Yue, and K. V. Rashmi. A large scale analysis of hundreds of in-memory cache clusters at Twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, 2020.